

**Implementasi dan Analisis Perbandingan Kinerja Algoritma Breadth-First Search
untuk Pathfinding pada Grid Menggunakan Open Multi-Processing dan Open
Computing Language**



Penyusun:

Silvia (25032014006)

Dosen Pengampu:

Harmon Prayogi, M.Sc. (0026069206)

UNIVERSITAS NEGERI SURABAYA

2025/2026

BAB I. PENDAHULUAN

1.1. Latar Belakang

Pencarian jalur (*pathfinding*) merupakan permasalahan komputasi yang pada sistem navigasi, di mana proses *pathfinding* bertujuan untuk menemukan jalur terbaik dari titik awal menuju tujuan pada suatu lingkungan [1]. Salah satu algoritma yang umum digunakan untuk menyelesaikan permasalahan *pathfinding* adalah *Breadth-First Search* (BFS), karena karakteristiknya yang menjamin penemuan lintasan terpendek pada graf tanpa bobot dengan menelusuri setiap simpul berdasarkan tingkat kedalamannya [2]. Namun, ketika ukuran *grid* yang digunakan semakin besar, jumlah ruang keadaan (*state space*) yang harus dieksplorasi meningkat, sehingga algoritma menghadapi tuntutan komputasi (*data-intensive*) dan kebutuhan alokasi memori yang tinggi [3], [4].

Pada arsitektur CPU konvensional, implementasi BFS umumnya dieksekusi secara sekuensial oleh satu alur eksekusi (*single-thread*) [4]. Ketika menangani *grid* berskala besar yang terdiri dari ratusan ribu hingga jutaan sel, proses ini cenderung menjadi *memory-bound* akibat tingginya perpindahan data antara unit pemrosesan dan memori serta rendahnya *data reuse*, sehingga menimbulkan *bottleneck* performa yang membatasi efisiensi komputasi dan pada akhirnya meningkatkan waktu eksekusi [5]. Oleh karena itu, diperlukan pemanfaatan teknik komputasi paralel untuk mempercepat proses pencarian jalur melalui pembagian beban kerja eksplorasi lapisan simpul (*frontier expansion*) ke beberapa unit pemrosesan yang dapat berjalan secara bersamaan.

Salah satu pendekatan yang dapat digunakan adalah komputasi heterogen (*heterogeneous computing*) yang memanfaatkan kemampuan pemrosesan dari CPU *multi-core* dan GPU dalam satu sistem komputasi. Pada proyek ini digunakan Open Multi-Processing (OpenMP) untuk mengelola *thread* pada arsitektur memori bersama (*shared memory*) [6]. Sementara itu, untuk menangani skalabilitas ekstrim *Open Computing Language* (OpenCL) digunakan karena memungkinkan eksekusi paralel pada berbagai perangkat melalui model berbasis kernel dan data-paralel [7]. Melalui pengujian ini, dilakukan analisis untuk mengevaluasi efisiensi masing-masing metode dalam meningkatkan kinerja algoritma BFS pada pencarian jalur *grid* berskala kecil hingga besar berdasarkan parameter waktu eksekusi, nilai speedup, efisiensi komputasi, serta pemanfaatan sumber daya komputasi, sehingga dapat diketahui metode yang paling efisien dan efektif di antara implementasi sekuensial, OpenMP, dan OpenCL [8].

1.2. Tujuan Proyek

Proyek ini bertujuan untuk mengimplementasikan dan menganalisis kinerja algoritma BFS pada pencarian jalur *grid* menggunakan pendekatan sekuensial, OpenMP, dan OpenCL. Selain itu, proyek ini juga berfokus pada evaluasi perbedaan performa antara ketiga metode

tersebut dalam konteks komputasi paralel dan heterogen. Tujuan utama dari proyek ini antara lain:

1. Mengimplementasikan algoritma BFS pada pencarian jalur *grid* dalam bentuk eksekusi sekuensial (*single-thread*) sebagai dasar pengukuran (*baseline* performa).
2. Mengimplementasikan algoritma BFS menggunakan OpenMP dan OpenCL.
3. Melakukan perbandingan kinerja di antara ketiga pendekatan tersebut berdasarkan parameter waktu eksekusi (*execution time*) pada berbagai ukuran *grid*.
4. Mengevaluasi nilai benchmarking yang diperoleh dari masing-masing metode paralel (OpenMP dan OpenCL).
5. Menganalisis efisiensi komputasi serta faktor-faktor non-linearitas perangkat keras untuk menentukan metode pemrosesan yang paling optimal dan efektif untuk kasus pencarian jalur pada *grid*.

BAB II. DASAR TEORI

2.1. Algoritma Breadth-First Search (BFS)

Breadth-First Search (BFS) merupakan algoritma penelusuran graf yang digunakan untuk menemukan lintasan terpendek (*shortest path*) pada graf tanpa bobot. BFS bekerja dengan mengunjungi simpul berdasarkan urutan jaraknya dari simpul sumber melalui pendekatan *level-by-level exploration*, dimulai dari simpul yang berjarak satu sisi (*edge*), kemudian dua sisi, dan seterusnya hingga tujuan ditemukan atau seluruh simpul yang dapat dijangkau telah dieksplorasi [9]. Algoritma BFS diimplementasikan menggunakan struktur data *queue* untuk mengelola simpul yang akan diproses, di mana pada setiap iterasi seluruh simpul pada lapisan pencarian saat ini (*frontier*) diperiksa tetangganya. Simpul yang belum pernah dikunjungi kemudian ditandai dan dimasukkan ke dalam *queue* untuk membentuk *frontier* pada iterasi berikutnya [10]. Karena setiap simpul pertama kali ditemukan melalui jalur dengan jumlah sisi minimum, BFS mampu menjamin penemuan lintasan terpendek dari titik awal menuju titik tujuan [9].

Konsep *frontier* memiliki peran penting dalam implementasi BFS paralel karena proses pemeriksaan tetangga dari setiap simpul pada *frontier* dapat dilakukan secara bersamaan oleh beberapa unit pemrosesan. Dengan memanfaatkan paralelisme, waktu eksekusi BFS dapat dikurangi secara signifikan, terutama pada graf berukuran besar. Namun, jumlah *frontier* yang berubah pada setiap iterasi dapat menyebabkan ketidakseimbangan beban kerja (*load imbalance*) dan *overhead* komputasi yang berpengaruh terhadap performa keseluruhan algoritma [10].

2.2. Konsep Parallelism & Arsitektur Heterogen

Parallelism dalam komputasi merujuk pada pendekatan di mana suatu pekerjaan dibagi menjadi beberapa bagian yang dapat dieksekusi secara bersamaan oleh lebih dari satu unit pemrosesan. Pendekatan ini digunakan untuk mengurangi waktu eksekusi, terutama pada permasalahan yang melibatkan data berukuran besar dan kebutuhan komputasi yang tinggi. Pada algoritma BFS, paralelisme dapat diterapkan pada proses eksplorasi *frontier*, karena setiap simpul pada *frontier* dapat memeriksa tetangganya secara independen, pekerjaan tersebut dapat didistribusikan ke beberapa unit pemrosesan sehingga proses pencarian dapat diselesaikan lebih cepat dibandingkan pendekatan sekuensial. Kebutuhan akan percepatan komputasi seperti ini semakin meningkat seiring berkembangnya aplikasi modern yang menuntut pengurangan waktu penyelesaian komputasi (*time-to-solution*) dan peningkatan efisiensi pemanfaatan sumber daya komputasi [11].

Pada arsitektur CPU *multi-core*, paralelisme umumnya diimplementasikan melalui model *thread-based execution* seperti yang digunakan pada OpenMP. Dalam model ini, beberapa *thread* bekerja secara bersamaan dalam ruang memori bersama (*shared memory*) untuk menyelesaikan bagian pekerjaan yang berbeda. Pembagian tugas dilakukan melalui mekanisme penjadwalan yang mendistribusikan iterasi atau kelompok data kepada setiap *thread*. Meskipun pendekatan ini relatif mudah diterapkan, pencapaian efisiensi dan skalabilitas yang tinggi tetap dipengaruhi oleh faktor seperti sinkronisasi antar *thread*, pembagian beban kerja, serta *overhead* yang muncul selama proses penjadwalan dan pengelolaan *runtime* [12].

Selain memanfaatkan CPU *multi-core*, komputasi paralel juga dapat diterapkan melalui arsitektur heterogen yang menggabungkan CPU sebagai *host* dan GPU sebagai *device*. Pada model ini, CPU bertugas untuk mengelola alur program, mempersiapkan data, dan inisialisasi eksekusi komputasi, sedangkan GPU menjalankan kernel yang berisi operasi paralel dalam jumlah besar. Eksekusi pada GPU dilakukan oleh ribuan thread yang diorganisasikan ke dalam struktur hierarkis berupa *grid* dan *block* pada CUDA atau NDRange dan *work-group* pada OpenCL, di mana *thread* dalam satu kelompok dapat melakukan sinkronisasi dan berbagi sumber daya memori tertentu. Selain hierarki *thread*, performa komputasi juga dipengaruhi oleh hierarki memori yang digunakan, mulai dari memori global berkapasitas besar hingga memori lokal yang memiliki latensi lebih rendah. Oleh karena itu, pembagian beban kerja yang tepat antara *host* dan *device* serta pengelolaan akses memori yang efisien menjadi faktor penting dalam mencapai peningkatan performa pada sistem komputasi heterogen, terutama mengingat adanya *overhead* transfer data antara CPU dan GPU yang dapat menjadi *bottleneck* pada aplikasi tertentu [11], [13].

2.3. CPU Parallelism dengan OpenMP

Open Multi-Processing (OpenMP) merupakan *Application Programming Interface* (API) untuk pemrograman paralel pada sistem *shared-memory* yang memungkinkan beberapa thread bekerja secara bersamaan dalam satu ruang memori yang sama. OpenMP mengadopsi model *fork-join*, yaitu program diawali oleh satu *thread* awal (*initial thread*) yang kemudian membuat sejumlah *thread* tambahan ketika memasuki wilayah paralel dan kembali bergabung

setelah pekerjaan selesai. Implementasi paralel pada OpenMP umumnya dilakukan menggunakan direktif berbasis **pragma**, sehingga pengembang dapat menambahkan kemampuan paralel tanpa mengubah struktur utama program secara signifikan. Salah satu direktif yang paling mendasar adalah **#pragma omp parallel**, yang digunakan untuk membentuk sekumpulan *thread* dan menjalankan blok kode tertentu secara bersamaan melalui beberapa *thread* yang kemudian dijadwalkan pada inti prosesor yang tersedia [14].

OpenMP menyediakan direktif **#pragma omp for** yang memungkinkan iterasi pada suatu perulangan (*loop*) didistribusikan secara otomatis ke beberapa *thread* sehingga beban kerja dapat dibagi secara lebih merata. Dalam pendekatan *frontier-based parallel* BFS, setiap *thread* bertanggung jawab memproses sebagian simpul pada *frontier* saat ini dan menghasilkan *frontier* lokal untuk iterasi berikutnya. Strategi ini memungkinkan eksplorasi banyak simpul dilakukan secara simultan sehingga waktu pencarian dapat dikurangi dibandingkan implementasi sekuensial. Efektivitas pendekatan tersebut bergantung pada kemampuan sistem dalam mendistribusikan pekerjaan secara seimbang (*load balancing*) agar seluruh *thread* dapat dimanfaatkan secara optimal [14].

Karena seluruh *thread* bekerja pada ruang memori yang sama, akses terhadap struktur data bersama berpotensi menimbulkan *race condition* ketika beberapa *thread* mencoba membaca atau memperbarui lokasi memori yang sama secara bersamaan. Pada implementasi BFS paralel, kondisi ini dapat terjadi ketika beberapa *thread* menemukan simpul yang sama dan berusaha memperbarui status *visited* secara serentak. Untuk menjaga konsistensi data, OpenMP menyediakan berbagai mekanisme sinkronisasi, termasuk operasi atomik seperti **#pragma omp atomic** dan **#pragma omp atomic capture**, yang dapat digunakan untuk memastikan pembaruan terhadap data bersama dilakukan secara aman oleh beberapa *thread*. Mekanisme ini membantu mencegah duplikasi pekerjaan dan memastikan hanya satu *thread* yang berhasil menandai suatu simpul sebagai *visited* sehingga duplikasi pemrosesan dapat diminimalkan. Selain itu, OpenMP *memory model* mendefinisikan aturan visibilitas dan sinkronisasi data antar *thread* sehingga perubahan yang dilakukan oleh suatu *thread* dapat diamati secara benar oleh *thread* lainnya, yang pada akhirnya menjamin ketepatan hasil komputasi paralel [14].

2.4. GPU Parallelism dengan OpenCL

Open Computing Language (OpenCL) merupakan kerangka pemrograman paralel untuk sistem komputasi heterogen yang memungkinkan pemanfaatan perangkat komputasi yang berbeda, seperti CPU dan GPU, dalam satu model pemrograman yang terintegrasi. Dalam OpenCL, *host* bertugas mengelola alur eksekusi program, alokasi memori, dan pengiriman perintah komputasi, sedangkan *device* menjalankan operasi paralel melalui kernel yang dieksekusi pada perangkat komputasi. OpenCL juga memiliki model memori yang memisahkan memori host dan memori *device*, termasuk *global*, *constant*, *local*, dan *private memory*, sehingga transfer data antar *host* dan *device* menjadi bagian penting dari eksekusi program. Pada sistem dengan GPU diskrit, data umumnya perlu dipindahkan melalui jalur interkoneksi seperti PCIe sebelum kernel dijalankan, sehingga pengelolaan transfer data dan

pemilihan pola akses memori menjadi faktor penting dalam menentukan performa akhir. Dengan demikian, model *host-device* pada OpenCL memungkinkan pembagian beban kerja yang efektif antara CPU dan GPU sesuai karakteristik masing-masing perangkat [15], [16].

Paralelisme pada OpenCL diwujudkan melalui unit eksekusi ringan yang disebut *work-item*, yang kemudian dikelompokkan ke dalam *work-group*, sedangkan ruang eksekusi keseluruhan kernel direpresentasikan sebagai NDRange. Setiap *work-item* menjalankan kernel yang sama tetapi pada data yang berbeda, sehingga OpenCL sangat cocok untuk masalah yang memiliki karakteristik *data parallelism* tinggi. Dalam konteks BFS, pendekatan ini memungkinkan setiap *work-item* memproses simpul pada *frontier* secara bersamaan dan mengeksplorasi tetangga yang dapat dikunjungi pada saat yang sama. Karena *work-item* dalam satu *work-group* dapat berkoordinasi lebih efisien melalui memori lokal dan sinkronisasi tingkat grup, OpenCL dapat memanfaatkan GPU untuk memproses banyak simpul secara simultan sehingga *throughput* komputasi berpotensi meningkat dibandingkan eksekusi sekuensial pada CPU [15], [17].

Untuk mendukung eksekusi paralel yang efisien, OpenCL menyediakan hierarki memori yang terdiri atas *global memory*, *local memory*, dan *private memory*. *Global memory* dapat diakses oleh seluruh *work-item* tetapi memiliki latensi akses yang relatif tinggi, sedangkan *local memory* umumnya memiliki latensi akses yang lebih rendah dibandingkan *global memory* pada banyak arsitektur GPU. Sementara itu, *private memory* bersifat eksklusif untuk setiap *work-item* dan digunakan untuk menyimpan variabel lokal selama eksekusi kernel. Karena banyak *work-item* dapat mengakses data yang sama secara bersamaan, OpenCL menyediakan mekanisme sinkronisasi seperti *barrier* yang digunakan untuk memastikan seluruh *work-item* dalam suatu *work-group* mencapai titik eksekusi yang sama sebelum melanjutkan proses komputasi, serta operasi atomik untuk mengelola akses aman terhadap data bersama. Operasi atomik memungkinkan pembacaan dan pembaruan nilai memori dilakukan secara aman tanpa menimbulkan *race condition*, sedangkan OpenCL *memory model* mengatur visibilitas dan urutan akses data antar *work-item* selama eksekusi kernel. Kombinasi antara hierarki memori, sinkronisasi, dan eksekusi paralel inilah yang memungkinkan OpenCL memanfaatkan sumber daya GPU secara optimal untuk mempercepat komputasi berskala besar [15].

2.5. Benchmarking

Benchmarking merupakan proses pengukuran dan evaluasi kinerja suatu sistem untuk mengetahui tingkat efisiensi dan efektivitas implementasi yang digunakan. Dalam komputasi paralel, *benchmarking* umumnya dilakukan dengan membandingkan waktu eksekusi (*execution time*) antara program sekuensial dan program paralel pada beban kerja yang sama. Hasil pengukuran tersebut digunakan untuk menilai sejauh mana teknik paralelisme mampu meningkatkan performa komputasi [18]. Pada pengujian ini, *benchmarking* dilakukan dengan mengukur waktu eksekusi algoritma BFS pada implementasi sekuensial, OpenMP, dan OpenCL menggunakan variasi ukuran grid yang berbeda. Parameter yang diamati meliputi

waktu eksekusi, nilai *speedup* pada implementasi OpenMP dan OpenCL, dan efisiensi komputasi pada implementasi OpenMP.

2.5.1. Speedup

Speedup menunjukkan tingkat percepatan yang diperoleh dari implementasi paralel dibandingkan dengan implementasi sekuensial. Secara umum, *speedup* menunjukkan seberapa besar peningkatan kinerja yang dihasilkan oleh penerapan paralelisme. Nilai *speedup* dihitung dengan:

$$S(N) = \frac{T(1)}{T(N)}$$

dengan:

N = Jumlah *thread* paralel atau unit pemrosesan

$T(1)$ = Waktu eksekusi program sekuensial

$T(N)$ = Waktu eksekusi program paralel

Nilai *speedup* yang lebih besar dari satu menunjukkan bahwa implementasi paralel memberikan peningkatan performa dibandingkan implementasi sekuensial. Namun, peningkatan jumlah unit pemrosesan tidak selalu menghasilkan percepatan yang bersifat linier karena masih terdapat bagian program yang dieksekusi secara sekuensial serta berbagai *overhead* yang muncul selama proses paralelisasi, seperti komunikasi data, sinkronisasi, dan manajemen tugas [19], [20].

2.5.2. Efficiency

Selain *speedup*, efisiensi paralel (*parallel efficiency*) digunakan untuk mengukur tingkat pemanfaatan sumber daya pemrosesan yang tersedia. Efisiensi menunjukkan seberapa efektif sejumlah unit pemrosesan bekerja dibandingkan kondisi ideal, yaitu ketika peningkatan jumlah prosesor menghasilkan percepatan yang sepenuhnya linier. Nilai efisiensi dinyatakan dengan:

$$E(N) = \frac{S(N)}{N}$$

Dengan:

- $S(N)$ = Nilai *speedup*
- N = Jumlah *thread* paralel atau unit pemrosesan

Nilai $E(N) = 1$ menunjukkan kondisi ideal, yaitu ketika peningkatan jumlah unit pemrosesan menghasilkan percepatan yang sepenuhnya linier. Dalam praktiknya, efisiensi umumnya menurun seiring bertambahnya jumlah unit pemrosesan akibat adanya *overhead* paralel, biaya sinkronisasi, komunikasi data, serta ketidakseimbangan beban kerja (*load imbalance*) [20].

2.6. Hukum Paralelisme

Peningkatan performa pada sistem paralel tidak selalu berbanding lurus dengan jumlah unit pemrosesan yang digunakan. Terdapat faktor-faktor seperti bagian program yang bersifat sekuensial, biaya komunikasi, serta *overhead* sinkronisasi yang dapat membatasi tingkat percepatan yang diperoleh. Untuk menjelaskan fenomena tersebut, digunakan dua model teoritis yang umum dalam komputasi paralel, yaitu Hukum Amdahl (*Amdahl's Law*) dan Hukum Gustafson (*Gustafson's Law*) [20].

2.6.1. Amdahl's Law

Hukum Amdahl menyatakan bahwa percepatan maksimum suatu program dibatasi oleh proporsi bagian program yang tidak dapat diparalelkan. Jika suatu program terdiri atas bagian sekuensial sebesar s dan bagian paralel sebesar p , maka *speedup* teoritis pada N unit pemrosesan dapat dihitung menggunakan persamaan:

$$S(N) = \frac{s + p}{s + \frac{p}{N}}$$

dengan:

- s = Fraksi sekuensial, bagian dari total *code* yang tidak diparalelkan
- p = Fraksi paralel, bagian dari beban kerja program yang bisa dieksekusi secara paralel
- N = Jumlah *thread* paralel atau unit pemrosesan

Persamaan tersebut menunjukkan bahwa meskipun jumlah unit pemrosesan terus ditingkatkan, percepatan maksimum tetap dibatasi oleh keberadaan bagian sekuensial program. Oleh karena itu, semakin besar nilai s , semakin rendah batas *speedup* yang dapat dicapai [20].

2.6.2. Gustafon's Law

Sementara itu, Hukum Gustafson (*Gustafson's Law*) menggambarkan kondisi ketika ukuran masalah dapat diperbesar seiring dengan bertambahnya sumber daya komputasi. Pada model ini, peningkatan jumlah prosesor dimanfaatkan untuk menyelesaikan beban kerja yang lebih besar dalam waktu yang relatif tetap. *Speedup* menurut Hukum Gustafson dinyatakan sebagai:

$$S(N) = \frac{s + p \times N}{s + p}$$

dengan:

- s = Fraksi sekuensial, bagian dari total *code* yang tidak diparalelkan
- p = Fraksi paralel, bagian dari beban kerja program yang bisa dieksekusi secara paralel
- N = Jumlah *thread* paralel atau unit pemrosesan

Berbeda dengan Hukum Amdahl yang berasumsi bahwa ukuran masalah tetap (*fixed-size problem*), Hukum Gustafson mengasumsikan bahwa ukuran masalah dapat meningkat seiring bertambahnya jumlah unit pemrosesan. Oleh karena itu, model ini sering digunakan untuk

menggambarkan skalabilitas aplikasi komputasi modern yang bersifat *data-intensive* dan memanfaatkan sumber daya paralel untuk menangani beban kerja yang semakin besar [20].

BAB III. PERANCANGAN SISTEM

3.1. Gambaran Sistem

Sistem yang telah dikembangkan bertujuan untuk mengimplementasikan dan membandingkan kinerja algoritma BFS pada pencarian jalur *grid* menggunakan tiga pendekatan komputasi, yaitu eksekusi sekuensial, paralel CPU dengan OpenMP, dan paralel GPU dengan OpenCL. Proses dimulai dengan pembuatan *grid* dua dimensi yang berisi sel kosong dan *obstacle*, kemudian ditentukan titik awal dan titik tujuan sebagai parameter pencarian jalur. Selanjutnya, algoritma BFS dijalankan menggunakan ketiga metode implementasi tersebut pada *grid* yang sama untuk memastikan konsistensi hasil pengujian.

Selama proses eksekusi, waktu komputasi masing-masing metode diukur menggunakan mekanisme *benchmarkin*. Hasil pengukuran kemudian digunakan untuk menghitung nilai *speedup* dan efisiensi komputasi sebagai indikator peningkatan performa yang diperoleh melalui paralelisme. Data hasil pengujian disimpan ke dalam *file* untuk divisualisasikan. Selain itu, lintasan terpendek yang ditemukan oleh BFS disimpan dalam bentuk koordinat dan divisualisasikan menggunakan Python sehingga *user* dapat melihat jalur hasil pencarian.

3.2. Arsitektur dan Alur Kerja Sistem (Flowchart)

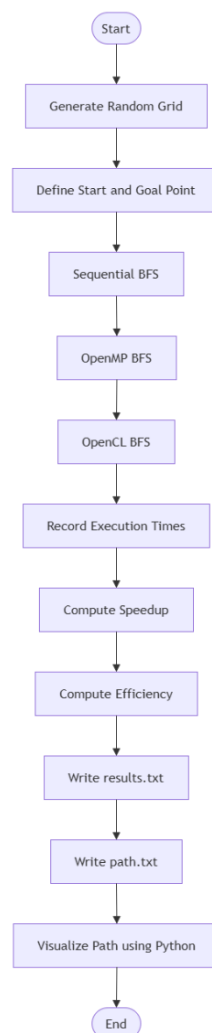
Untuk memberikan gambaran mengenai proses eksekusi sistem yang dikembangkan, digunakan beberapa *flowchart* yang merepresentasikan tahapan utama program serta mekanisme paralelisme yang diterapkan pada OpenMP dan OpenCL. *flowchart* ini digunakan untuk menjelaskan aliran data, pembagian beban kerja, serta interaksi antara CPU dan GPU selama proses *pathfinding* dan *benchmarking*. Secara umum, alur kerja sistem ini dapat diringkas sebagai berikut:

1. Membuat *grid* dengan ukuran tertentu.
2. Menentukan titik *start* dan *goal* pada *grid*.
3. Menjalankan BFS sekuensial.
4. Menjalankan OpenMP BFS.
5. Menjalankan OpenCL BFS.
6. Mengukur waktu eksekusi masing-masing implementasi.
7. Menghitung nilai *speedup* dan *efficiency* komputasi.
8. Menyimpan hasil pengujian ke dalam *file* keluaran.

9. Memvisualisasikan lintasan terpendek yang ditemukan.

Untuk menjabarkan proses tersebut secara detail, arsitektur sistem direpresentasikan melalui tiga *flowchart* utama. *Flowchart* tersebut mencakup alur program utama (*main.cpp*), mekanisme paralelisme pada implementasi OpenMP, serta alur eksekusi OpenCL pada arsitektur heterogen CPU-GPU. Melalui representasi ini, dapat dipahami bagaimana data diproses, bagaimana beban kerja didistribusikan ke berbagai unit pemrosesan, serta bagaimana setiap pendekatan BFS diimplementasikan dan dievaluasi dalam proses *benchmarking*. Berikut merupakan *flowchart* yang digunakan dalam perancangan sistem.

3.2.1. Flowchart Program Utama



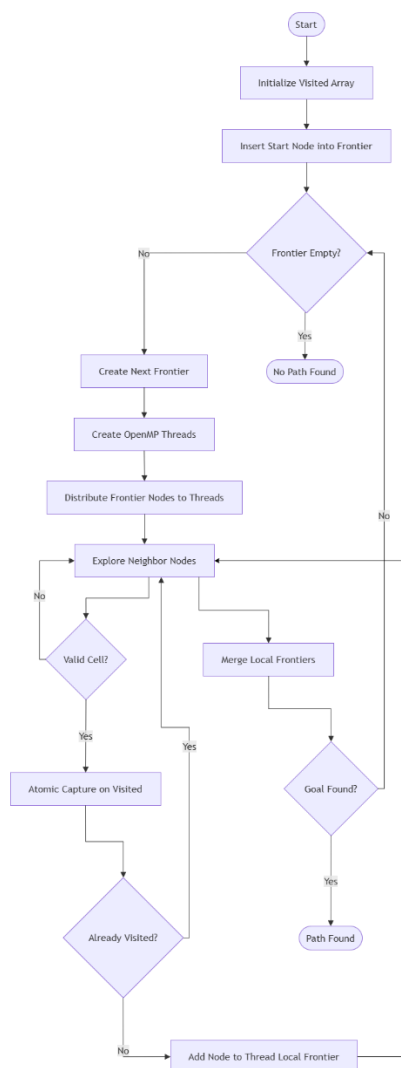
Gambar 1. Flowchart Program Utama (main.cpp)

Flowchart ini menunjukkan alur kerja program utama (*main.cpp*) yang digunakan untuk melakukan pengujian dan *benchmarking* algoritma BFS. Proses diawali dengan pembuatan *grid* beserta penentuan titik awal dan tujuan yang akan digunakan pada seluruh metode pengujian. Setelah proses inisialisasi selesai, sistem menjalankan algoritma BFS secara berurutan menggunakan tiga pendekatan yang berbeda, yaitu BFS sekuensial, OpenMP BFS,

dan OpenCL BFS. Waktu eksekusi masing-masing implementasi diukur untuk memperoleh data performa yang dapat digunakan dalam analisis.

Selanjutnya, sistem menghitung nilai *speedup* dan efisiensi komputasi berdasarkan hasil pengukuran waktu eksekusi. Data hasil pengujian kemudian disimpan ke dalam *file results.txt*, sedangkan lintasan terpendek yang ditemukan oleh BFS disimpan ke dalam *file path.txt*. Pada tahap akhir, data lintasan tersebut divisualisasikan menggunakan Python sehingga path yang ditemukan dapat diamati secara grafis.

3.2.2. Flowchart Implementasi BFS dengan OpenMP



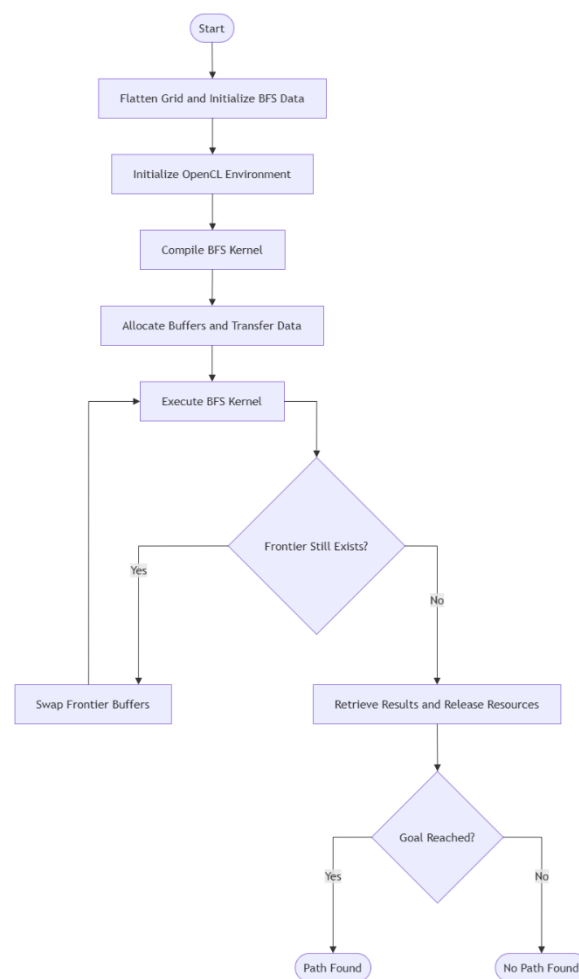
Gambar 2. Flowchart Implementasi OpenMP

Flowchart kedua ini menunjukkan mekanisme paralelisme BFS menggunakan OpenMP pada arsitektur CPU *multi-core*. Proses dimulai dengan inisialisasi struktur data *visited* dan pembentukan *frontier* awal yang berisi titik *start*. Selama *frontier* masih memiliki simpul yang harus dieksplorasi, sistem akan membentuk *frontier* baru untuk menampung simpul-simpul yang ditemukan pada iterasi berikutnya.

OpenMP kemudian membuat beberapa *thread* yang bertugas untuk memproses simpul-simpul pada *frontier* secara paralel. Setiap *thread* mengeksplorasi tetangga dari simpul yang menjadi tanggung jawabnya dan melakukan pemeriksaan posisi pada *grid*. Untuk mencegah terjadinya *race condition*, proses penandaan status *visited* dilakukan menggunakan operasi atomik, sehingga hanya satu *thread* yang dapat menandai suatu simpul sebagai telah dikunjungi.

Simpul yang berhasil ditemukan dan belum pernah dikunjungi akan dimasukkan ke dalam *thread-local frontier* milik masing-masing *thread*. Setelah seluruh *thread* menyelesaikan pekerjaannya, seluruh *local frontier* digabungkan menjadi *frontier* baru. Proses ini terus berulang sampai titik tujuan ditemukan atau tidak terdapat lagi simpul yang dapat dieksplorasi.

3.2.3. Flowchart Implementasi BFS dengan OpenCL



Gambar 3. Flowchart Implementasi OpenCL

Flowchart ketiga ini menggambarkan alur eksekusi BFS menggunakan OpenCL pada arsitektur komputasi heterogen berbasis GPU. Tahap awal dimulai dengan proses *flattening*, yaitu mengubah *grid* dua dimensi menjadi *array* satu dimensi untuk mempermudah pengelolaan data pada memori GPU. Setelah itu, struktur data BFS diinisialisasi sebelum proses komputasi dimulai.

Selanjutnya, lingkungan OpenCL dipersiapkan dengan melakukan inisialisasi platform, perangkat komputasi, *context*, dan *command queue*. Program OpenCL kemudian dikompilasi menjadi kernel yang akan dijalankan pada GPU. Setelah proses kompilasi selesai, sistem mengalokasikan *buffer* memori pada perangkat dan mentransfer data yang diperlukan dari CPU ke GPU.

Kernel BFS kemudian dieksekusi secara paralel oleh sejumlah besar *work-item* pada GPU untuk mengeksplorasi simpul-simpul pada *frontier*. Selama masih terdapat *frontier* baru yang dihasilkan, *buffer frontier* akan dirtukar dan kernel dijalankan kembali pada iterasi berikutnya. Setelah proses eksplorasi selesai, hasil komputasi dibaca kembali ke CPU dan seluruh sumber daya OpenCL dibebaskan. Tahap terakhir adalah melakukan pemeriksaan terhadap status simpul tujuan untuk menentukan apakah lintasan berhasil ditemukan atau tidak.

3.3. Input Sistem

3.3.1. Ukuran Grid

Input yang digunakan dalam sistem ini terdiri atas beberapa parameter yang menentukan karakteristik lingkungan pencarian serta konfigurasi eksekusi algoritma. Parameter utama berupa ukuran *grid* yang direpresentasikan melalui jumlah baris (*rows*) dan kolom (*cols*) pada matriks dua dimensi. Dimensi tersebut menentukan ukuran ruang pencarian (*state space*) yang harus dieksplorasi oleh algoritma BFS, sehingga semakin besar ukuran grid maka semakin banyak simpul yang harus diproses selama pencarian jalur. Pada proses pengujian, ukuran *grid* divariasikan dari skala kecil hingga besar untuk mengevaluasi pengaruh peningkatan beban kerja terhadap performa implementasi sekuensial, OpenMP, dan OpenCL.

3.3.2. Obstacle Density

Selain dimensi *grid*, sistem juga menerima parameter berupa probabilitas kemunculan *obstacle*. Parameter ini digunakan untuk menentukan distribusi sel yang dapat dilalui dan sel yang berperan sebagai rintangan. Nilai probabilitas tersebut mengontrol tingkat kompleksitas lingkungan pencarian, karena semakin tinggi kerapatan rintangan maka semakin banyak jalur yang harus dievaluasi oleh algoritma BFS untuk menemukan lintasan menuju tujuan.

3.3.3. Titik Awal dan Titik Tujuan

Setelah *grid* berhasil dibuat, sistem menetapkan titik awal dan titik tujuan yang menjadi dasar proses pencarian jalur. Pada implementasi ini, titik awal ditempatkan pada koordinat pojok kiri atas *grid*, yaitu (0,0), sedangkan titik tujuan ditempatkan pada koordinat pojok kanan bawah, yaitu (*rows*-1, *cols*-1). Kedua koordinat tersebut digunakan sebagai sumber dan target pencarian lintasan terpendek oleh seluruh implementasi BFS sehingga hasil yang diperoleh dapat dibandingkan secara konsisten.

3.4. Output Sistem

3.4.1. Output pada Terminal

Ketika program utama dieksekusi, sistem menampilkan berbagai informasi hasil pengujian secara langsung melalui terminal. Informasi tersebut digunakan untuk memantau proses eksekusi serta memberikan ringkasan performa dari setiap implementasi BFS yang diuji. Selama proses berlangsung, sistem menampilkan status eksekusi untuk menunjukkan metode yang sedang dijalankan. Setelah seluruh proses pengujian selesai, sistem akan menampilkan waktu eksekusi (*execution time*) dari masing-masing metode dalam satuan detik. Selain itu, sistem juga menghitung dan menampilkan nilai *speedup* untuk implementasi OpenMP dan OpenCL, dan nilai *efficiency* untuk implementasi OpenMP.

3.4.2. File Output Data

Selain menampilkan hasil pada terminal, sistem juga menghasilkan *file* dalam format *.txt* yang digunakan sebagai media penyimpanan data hasil pengujian dan visualisasi. *File* pertama adalah *results.txt*, yang berfungsi sebagai penyimpanan data benchmarking dari seluruh implementasi BFS yang diuji. *File* ini memuat waktu eksekusi implementasi sekuensial, OpenMP, dan OpenCL, nilai *speedup* OpenMP dan OpenCL, an juga nilai *efficiency* OpenMP yang dihitung berdasarkan hasil pengujian. Data yang tersimpan pada *results.txt* kemudian digunakan sebagai sumber utama dalam pembuatan grafik perbandingan performa, grafik *speedup*, dan grafik efisiensi pada tahap visualisasi.

File kedua adalah *path.txt*, yang dihasilkan ketika algoritma BFS berhasil menemukan lintasan menuju titik tujuan. *File* ini menyimpan informasi mengenai lingkungan pencarian beserta hasil *path* yang ditemukan. Struktur data di dalam berkas dimulai dengan informasi ukuran *grid*, diikuti representasi seluruh matriks *grid* yang berisi nilai 0 untuk sel yang dapat dilalui dan nilai 1 untuk *obstacle*. Pada bagian akhir *file*, disimpan daftar koordinat yang membentuk *path* terpendek hasil proses *backtracking* dari titik tujuan menuju titik awal. Data tersebut kemudian digunakan untuk menghasilkan visualisasi jalur.

3.4.3. Visualisasi Grafik

Output grafik dihasilkan melalui pemrosesan *file* *path.txt* dan *results.txt* menggunakan Bahasa pemrograman **Python** dengan *library* **Matplotlib**. Visualisasi ini bertujuan untuk mempermudah interpretasi hasil pencarian jalur serta analisis performa dari ketiga metode yang diuji.

Visualisasi pertama berupa tampilan lintasan terpendek yang menunjukkan representasi *grid* secara dua dimensi. Pada visualisasi tersebut, *obstacle* ditampilkan dengan warna yang berbeda dari area yang dapat dilalui, sedangkan titik awal dan tujuan diberikan warna khusus agar mudah diidentifikasi. Lintasan terpendek yang ditemukan oleh algoritma BFS kemudian digambarkan sebagai rangkaian koordinat yang menghubungkan titik awal dan titik tujuan. Selain menampilkan hasil akhir, visualisasi juga menyediakan animasi yang memperlihatkan perkembangan lintasan sepanjang proses penelusuran jalur. Visualisasi kedua berupa *bar char* yang menampilkan perbandingan waktu eksekusi antara BFS sekuensial, OpenMP BFS, dan OpenCL BFS. Selain itu, ditampilkan juga grafik nilai *speedup* untuk implementasi OpenMP dan OpenCL, serta grafik *efficiency* untuk implementasi OpenMP.

3.5. Keputusan Desain

3.5.1. Pemilihan Algoritma Breadth-First Search (BFS)

Algoritma BFS dipilih karena mampu menjamin penemuan lintasan terpendek pada graf tanpa bobot. Pada representasi *grid* yang digunakan dalam proyek ini, setiap perpindahan antar sel memiliki biaya yang sama sehingga BFS menjadi solusi yang tepat untuk menemukan jalur terpendek antara titik awal dan titik tujuan. Selain memiliki kompleksitas yang relatif sederhana, BFS juga memiliki karakteristik *frontier-based exploration* yang memungkinkan proses eksplorasi simpul pada level yang sama dilakukan secara paralel menggunakan OpenMP maupun OpenCL [9].

3.5.2. Perataan Matriks Grid (Flattening Grid)

Pada implementasi OpenCL, representasi *grid* dua dimensi diubah menjadi *array* satu dimensi sebelum dikirim ke perangkat GPU. Proses ini dilakukan menggunakan transformasi indeks:

$$[index = row \times cols + column]$$

sehingga setiap elemen *grid* dapat diakses melalui satu indeks linear. Pendekatan ini dipilih karena OpenCL bekerja lebih efisien dengan data yang tersimpan secara *contiguous* dalam memori. Selain meningkatkan *locality* akses memori, representasi satu dimensi juga menyederhanakan proses alokasi *buffer* dan transfer data dari CPU ke GPU. Dengan demikian, *overhead* pengelolaan data pada GPU dapat dikurangi dibandingkan penggunaan struktur data dua dimensi yang lebih kompleks [21].

3.5.3. Representasi Frontier Array/Bitmap pada OpenCL

Berbeda dengan implementasi BFS sekuensial yang menggunakan struktur data *queue*, implementasi OpenCL merepresentasikan *frontier* sebagai *array bitmap* yang berisi nilai biner, di mana nilai 1 menunjukkan bahwa suatu simpul berada pada *frontier* aktif, sedangkan nilai 0 menunjukkan bahwa simpul tersebut tidak termasuk dalam *frontier* saat ini.

Pendekatan ini dipilih karena *queue* bersifat dinamis dan sulit dikelola secara efisien oleh ribuan work-item yang berjalan secara bersamaan pada GPU. Dengan menggunakan bitmap frontier, setiap work-item dapat langsung memeriksa status suatu simpul berdasarkan indeks globalnya tanpa memerlukan operasi enqueue atau dequeue yang kompleks. Strategi ini memungkinkan proses eksplorasi frontier dilakukan secara lebih sederhana dan sesuai dengan model data *parallelism* yang digunakan OpenCL [22].

3.5.4. Pengolahan Thread-Local Frontier pada OpenMP

Pada implementasi OpenMP, setiap *thread* diberikan *frontier* lokal untuk menyimpan simpul baru yang ditemukan selama proses eksplorasi. Setelah seluruh *thread* menyelesaikan pekerjaannya, *frontier* lokal tersebut digabungkan menjadi *frontier global* yang digunakan pada iterasi berikutnya.

Pendekatan ini dipilih untuk mengurangi konflik akses terhadap struktur data bersama. Apabila seluruh *thread* langsung menambahkan simpul ke satu *frontier global* yang sama, maka akan terjadi peningkatan *overhead* sinkronisasi yang dapat menurunkan performa paralel. Dengan menggunakan *frontier* lokal, sebagian besar operasi penambahan simpul dapat dilakukan tanpa sinkronisasi sehingga efisiensi pemrosesan *multi-thread* menjadi lebih baik.

3.5.5. Penggunaan Operasi Atomik untuk Konflik Data

Baik pada implementasi OpenMP maupun OpenCL, beberapa *thread* atau *work-item* dapat menemukan simpul yang sama secara bersamaan. Kondisi tersebut berpotensi menimbulkan *race condition* apabila beberapa unit pemrosesan mencoba memperbarui status *visited* pada lokasi memori yang sama secara bersamaan.

Untuk mengatasi permasalahan tersebut digunakan operasi atomik. Pada implementasi OpenMP digunakan `#pragma omp atomic capture`, sedangkan pada implementasi OpenCL digunakan fungsi `atomic_cmpxchg()`. Mekanisme ini memastikan bahwa hanya satu *thread* atau *work-item* yang berhasil menandai suatu simpul sebagai *visited* dan menambahkannya ke *frontier* berikutnya. Dengan demikian, duplikasi pekerjaan dapat dihindari dan konsistensi data selama proses komputasi paralel tetap terjaga [14], [15].

3.5.6. Pola Akses Memori dan Sinkronisasi pada OpenCL

Implementasi OpenCL menggunakan pola akses memori berbasis indeks global, di mana setiap *work-item* bertanggung jawab memproses satu simpul tertentu pada *frontier* aktif. Seluruh *work-item* mengakses data *grid*, *visited*, *frontier*, dan *next_frontier* melalui *global memory* yang dapat diakses oleh seluruh unit eksekusi pada perangkat GPU.

Selain itu, digunakan mekanisme sinkronisasi berupa *barrier* pada beberapa tahap eksekusi kernel untuk memastikan seluruh *work-item* menyelesaikan pekerjaannya sebelum berpindah ke fase berikutnya. Sinkronisasi ini diperlukan terutama ketika *frontier* baru telah selesai dibentuk dan akan digunakan sebagai *frontier* aktif pada iterasi BFS berikutnya. Penggunaan *barrier* membantu menjaga konsistensi data antar *work-item* dan mencegah terjadinya pembacaan data yang belum selesai diperbarui oleh *work-item* lain [15].

BAB IV. IMPLEMENTASI SISTEM

4.1. Lingkungan Pengembangan (Environment Setup)

4.1.1. Spesifikasi Hardware

Pengujian dilakukan menggunakan laptop dengan spesifikasi yang mendukung pemrosesan paralel pada CPU dan GPU. Perangkat yang digunakan adalah ASUS TUF Gaming A16 (FA608PP) dengan spesifikasi:

Komponen	Spesifikasi
Model	ASUS TUF Gaming A16 (FA608PP)

Prosesor	AMD Ryzen 9 8940HX
Base Clock	2.40 GHz
Jumlah Core	16 Cores
Jumlah Thread	32 Threads
GPU 1	NVIDIA GeForce RTX 5070 (8GB GDDR7)
Integrated GPU	AMD Radeon™ 610M
RAM	16 GB DDR5
Storage	1 TB PCIe® 4.0 NVMe™ M.2 SSD

Tabel 1. Spesifikasi Hardware

4.1.2. Spesifikasi Software

Software yang digunakan berfungsi untuk menyediakan lingkungan pengembangan, kompilasi program, *runtime* paralel, serta visualisasi hasil pengujian. Sistem dijalankan pada Windows 11 Home 64-bit dengan *compiler* GCC/G++ versi 15.2.0. Implementasi paralel CPU memanfaatkan OpenMP versi 4.5, sedangkan implementasi GPU menggunakan OpenCL versi 2.1 yang didukung oleh *driver* AMD. Selain itu, proses visualisasi hasil pencarian jalur dan *benchmarking* dilakukan menggunakan **Python** melalui **Jupyter Notebook**.

Komponen	Spesifikasi
Operating System	Windows 11
Compiler C++	GCC/G++ 15.2.0 (MSYS2)
OpenMP Runtime	OpenMP 4.5
OpenCL SDK & Driver	OpenCL 2.1 AMD-APP
Python Version	Python 3.13.6
Environment Visualisasi	Jupyter Notebook

Tabel 2. Spesifikasi Software

4.2. Implementasi OpenMP

Implementasi paralel pada sisi CPU dilakukan menggunakan OpenMP dengan pendekatan *frontier-based* BFS. Tujuan utama implementasi ini adalah untuk memanfaatkan kemampuan *multi-core processor* untuk mempercepat proses eksplorasi simpul pada setiap level BFS. Berbeda dengan BFS sekuensial yang memproses setiap simpul secara berurutan menggunakan satu antrian (*queue*), implementasi OpenMP membagi pekerjaan eksplorasi simpul kepada beberapa *thread* yang berjalan secara bersamaan.

Pada awal eksekusi, sistem menginisialisasi matriks *visited* untuk mencatat simpul yang telah dikunjungi serta membuat *frontier* yang berisi titik awal.

```
vector<vector<int>> visited(g.rows, vector<int>(g.cols, 0));

vector<Point> frontier;
frontier.push_back(g.start);
visited[g.start.x][g.start.y] = 1;
```

Untuk meningkatkan performa paralel, setiap *thread* diberikan sebuah *thread-local frontier* yang digunakan untuk menyimpan simpul baru hasil eksplorasi. Pendekatan ini dipilih untuk mengurangi kebutuhan sinkronisasi saat beberapa *thread* menambahkan simpul secara bersamaan.

```
int max_threads = omp_get_max_threads(); vector<vector<Point>>
thread_local frontiers(max_threads);
```

Proses paralelisasi dilakukan menggunakan **#pragma omp parallel** dan **#pragma omp for**. Melalui mekanisme ini, simpul-simpul yang berada pada *frontier* saat ini didistribusikan secara otomatis ke seluruh *thread* yang tersedia sehingga proses eksplorasi dapat dilakukan secara bersamaan.

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();

    #pragma omp for nowait
    for (int i = 0; i < (int)frontier.size(); i++){
        ...
    }
}
```

Karena beberapa *thread* dapat menemukan simpul yang sama secara bersamaan, digunakan operasi atomik untuk menjaga konsistensi data pada matriks *visited* dan mencegah terjadinya *race condition*.

```
#pragma omp atomic capture
{
    already_visited = visited[nx][ny];
    visited[nx][ny] = 1;
}
```

Apabila simpul belum pernah dikunjungi, simpul tersebut akan dimasukkan ke dalam *frontier* lokal milik *thread* yang menemukannya. Setelah seluruh *thread* selesai bekerja, seluruh *frontier* lokal kemudian digabungkan menjadi *frontier* global baru yang akan digunakan pada iterasi BFS berikutnya [14].

Melalui pendekatan OpenMP tersebut, proses eksplorasi simpul dapat dilakukan secara paralel oleh banyak *thread* CPU. Penggunaan *thread-local frontier* membantu mengurangi *overhead* sinkronisasi, sedangkan operasi atomik memastikan konsistensi data selama proses pencarian berlangsung. Kombinasi kedua teknik tersebut memungkinkan implementasi OpenMP memperoleh performa yang lebih baik dibandingkan BFS sekuensial pada *grid* berukuran besar.

4.3. Implementasi OpenCL

Implementasi paralel pada sisi GPU dilakukan menggunakan OpenCL dengan pendekatan *frontier-based Breadth-First Search* (BFS). Berbeda dengan OpenMP yang memanfaatkan beberapa *thread* CPU, OpenCL memanfaatkan ribuan *work-item* yang dapat dieksekusi secara paralel oleh GPU untuk mempercepat proses eksplorasi simpul pada *grid* berukuran besar.

Sebelum data dikirim ke GPU, representasi *grid* dua dimensi diubah menjadi *array* satu dimensi, karena GPU bekerja lebih efisien dengan data yang tersimpan secara linear di memori serta mempermudah proses transfer data antara *host* dan *device*.

```
h_grid_flat[i * g.cols + j] = g.grid[i][j];
```

Selain *grid*, sistem juga menginisialisasi struktur data *visited* dan *frontier* yang digunakan selama proses BFS. Berbeda dengan implementasi sekuensial yang menggunakan *queue*, implementasi OpenCL merepresentasikan *frontier* sebagai *array bitmap*. Setiap elemen *array* menunjukkan apakah suatu simpul termasuk ke dalam *frontier* yang sedang diproses.

```
h_frontier[start_idx] = 1;  
h_visited[start_idx] = 1;
```

Setelah data dipersiapkan, sistem melakukan inisialisasi lingkungan OpenCL yang meliputi pemilihan platform, pemilihan perangkat GPU, pembuatan *context*, *command queue*, serta kompilasi kernel yang akan dieksekusi pada *device*.

```
cl_int err;  
cl_platform_id platform;  
clGetPlatformIDs(1, &platform, NULL);  
  
cl_device_id device;  
clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL);  
  
cl_context context = clCreateContext(NULL, 1, &device, NULL, NULL, &err);  
cl_command_queue queue = clCreateCommandQueueWithProperties(context, device, 0, &err);
```

Data yang telah disiapkan kemudian dipindahkan ke memori GPU melalui sejumlah *buffer* OpenCL yang digunakan untuk menyimpan *grid*, *visited*, *frontier*, dan *next frontier*.

```
cl_mem d_grid;  
cl_mem d_visited;  
cl_mem d_frontier;  
cl_mem d_next_frontier;
```

Proses pencarian jalur dilakukan oleh kernel **persistent_bfs_kernel**. Setiap *work-item* bertanggung jawab memeriksa satu sel pada *grid* berdasarkan nilai *global ID* yang dimilikinya. Apabila sel tersebut berada pada *frontier* aktif, *work-item* akan mengeksplorasi tetangga yang berada di sekitarnya.

```
int id = get_global_id(0);
```

Karena beberapa *work-item* dapat menemukan simpul yang sama secara bersamaan, digunakan operasi atomik untuk menjaga konsistensi data pada *array visited*.

```
atomic_cmpxchg(&visited[nid], 0, 1);
```

Operasi atomik tersebut memastikan bahwa setiap simpul hanya dapat ditandai sebagai telah dikunjungi satu kali sehingga *race condition* dapat dihindari. Selain itu, kernel juga menggunakan instruksi *barrier* untuk melakukan sinkronisasi antar *work-item* pada tahap-tahap tertentu selama proses BFS berlangsung.

```
barrier(CLK_GLOBAL_MEM_FENCE);
```

Eksekusi kernel dilakukan secara berulang hingga tidak terdapat *frontier* baru yang dapat dieksplorasi atau titik tujuan berhasil ditemukan. Setelah proses selesai, hasil pencarian dikembalikan ke *host* dan seluruh *resource* OpenCL yang digunakan dilepaskan kembali [15].

4.4. Implementasi Visualisasi dengan Python

Visualisasi hasil implementasi dilakukan menggunakan bahasa pemrograman **Python** dengan *library* **Matplotlib**. Tahap ini bertujuan untuk mempermudah interpretasi hasil pencarian jalur serta analisis performa dari algoritma BFS yang telah diimplementasikan. Data visualisasi diperoleh dari *file* *path.txt* dan *results.txt*.

Berdasarkan data pada *path.txt*, sistem membangun representasi visual *grid* dan menampilkan lintasan terpendek yang telah ditentukan oleh BFS. Selain menghasilkan tampilan akhir jalur pencarian, sistem juga menampilkan animasi yang memperlihatkan proses pembentukan lintasan dari titik awal menuju titik tujuan. Visualisasi ini membantu memverifikasi bahwa jalur yang ditemukan telah sesuai dengan kondisi *grid* dan *obstacle* yang digunakan selama pengujian.

Selanjutnya, data pada *results.txt* digunakan untuk menghasilkan grafik performa dalam bentuk *bar chart*. Visualisasi yang dihasilkan meliputi perbandingan waktu eksekusi antara BFS sekuensial, OpenMP BFS, dan OpenCL BFS, serta grafik nilai *speedup* dan *efficiency* dari implementasi paralel. Dengan adanya visualisasi tersebut, hasil pengujian dapat dianalisis secara lebih mudah dibandingkan dengan hanya menggunakan data numerik yang ditampilkan pada terminal.

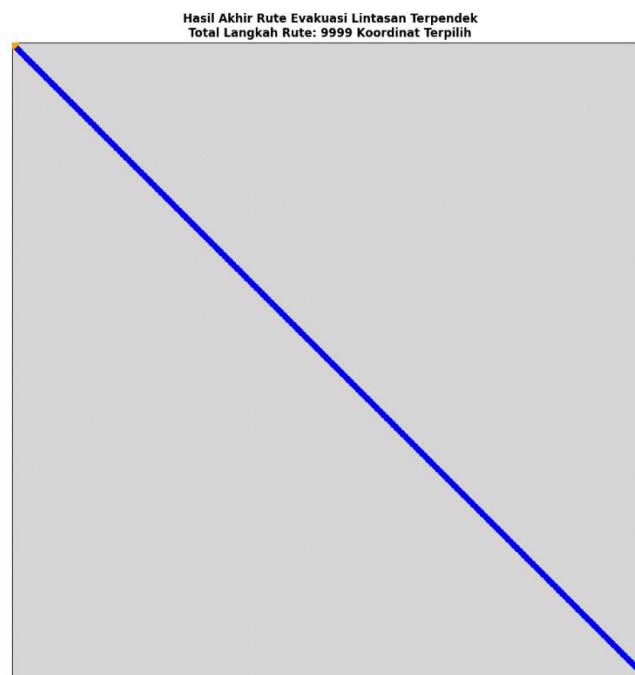
BAB V. ANALISIS DATA PENGUJIAN

5.1. Skenario Pengujian dan Analisis Output Sistem

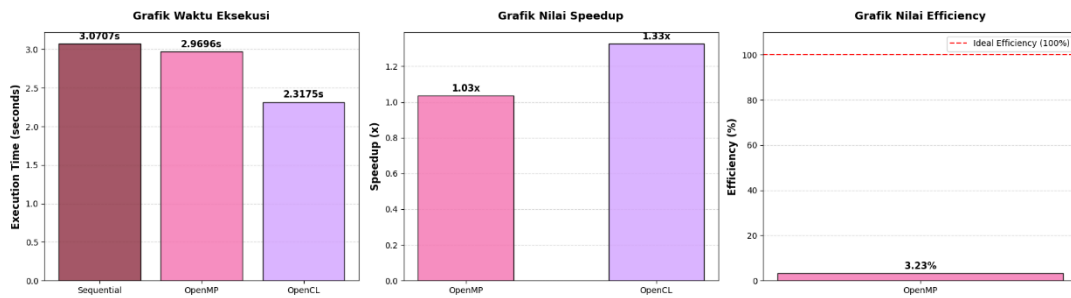
Pengujian dilakukan untuk mengevaluasi kinerja algoritma BFS yang diimplementasikan menggunakan tiga pendekatan komputasi, yaitu sekuensial BFS, OpenMP BFS, dan OpenCL BFS. Seluruh implementasi diuji menggunakan lingkungan *hardware* dan *software* seperti yang telah dijelaskan pada Bab IV. Pada setiap pengujian, ketiga metode menggunakan data *grid* yang hamper sama sehingga perbedaan hasil yang diperoleh hanya dipengaruhi oleh mekanisme komputasi yang digunakan. Parameter yang diamati meliputi waktu eksekusi (*execution time*), nilai *speedup*, dan efisiensi komputasi untuk menilai tingkat percepatan yang dihasilkan oleh implementasi paralel dibandingkan implementasi sekuensial.

Untuk menganalisis pengaruh ukuran masalah terhadap performa sistem, pengujian dilakukan pada beberapa variasi dimensi *grid*, yaitu 1000×1000 , 2000×2000 , 3000×3000 , 4000×4000 , 5000×5000 , 10000×10000 , 15000×15000 , dan 20000×20000 . Setiap *grid* dibuat secara acak dengan *obstacle density* sebesar 25%. Variasi ukuran *grid* tersebut dipilih untuk merepresentasikan beban kerja mulai dari skala kecil hingga skala sangat besar sehingga dapat diamati bagaimana perubahan jumlah simpul yang harus dieksplorasi memengaruhi performa masing-masing metode.

Output sistem terdiri dari dua jenis hasil utama, yaitu hasil *benchmarking* dan hasil visualisasi lintasan. Sistem menghasilkan *file path.txt* yang menyimpan representasi *grid* serta koordinat lintasan terpendek yang ditemukan oleh algoritma BFS. Data tersebut divisualisasikan menggunakan Python sehingga pengguna dapat mengamati jalur yang dihasilkan. Gambar dibawah ini merupakan pengujian dengan ukuran *grid* 5000×5000 :



Gambar 4. Hasil Pathfinding BFS untuk Grid Berukuran 5000×5000



Gambar 5. Hasil Benchmarking untuk Grid Berukuran 5000×5000

Pada pengujian *grid* 5000×5000 , terjadi pergeseran peta performa yang krusial antar metode komputasi. Metode OpenCL (GPU) berhasil mencatatkan waktu eksekusi tercepat yaitu 2,3175 detik, mengungguli metode sekuensial (3.0707 detik) dan OpenMP (2.9696 detik).

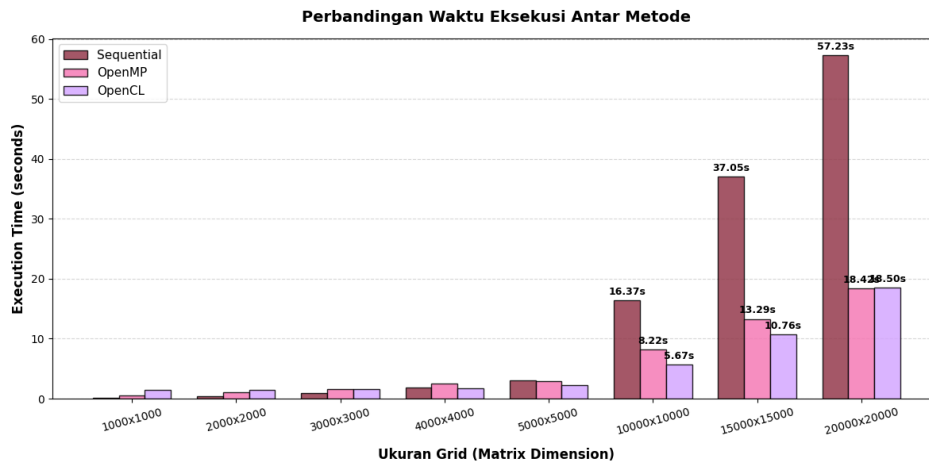
5.2. Perbandingan Data Kinerja (Benchmark)

5.2.1. Hasil dan Analisis Waktu Eksekusi

Pengujian waktu eksekusi dilakukan untuk mengukur performa absolut dari masing-masing metode, yaitu sekuensial, OpenMP (multi-core CPU), dan OpenCL (GPU). Pengujian ini menggunakan berbagai ukuran *grid* mulai dari 1000×1000 hingga 20000×20000 dengan obstacle density yang sama, yaitu 25% untuk melihat bagaimana beban kerja memengaruhi waktu pemrosesan.

Ukuran Grid	Sekuensial	OpenMP	OpenCL
1000×1000	0.0993	0.5088	1.4793
2000×2000	0.4253	1.0549	1.5213
3000×3000	0.9247	1.6069	1.6205
4000×4000	1.8165	2.5576	1.7614
5000×5000	3.0707	2.9696	2.3175
10000×10000	16.3672	8.2206	5.6704
15000×15000	37.0478	13.2905	10.7635
20000×20000	57.2313	18.4173	18.5031

Tabel 3. Hasil Waktu Eksekusi



Gambar 5. Grafik Perbandingan Waktu Eksekusi Antar Metode

Berdasarkan Tabel 3 dan Gambar 5, waktu eksekusi seluruh implementasi meningkat seiring bertambahnya ukuran *grid* yang digunakan. Pada ukuran *grid* kecil hingga menengah (1000×1000 sampai 5000×5000), implementasi sekuensial menunjukkan performa terbaik dengan waktu eksekusi yang lebih rendah dibandingkan OpenMP maupun OpenCL. Hal ini terjadi karena overhead yang muncul dari pembentukan thread pada OpenMP serta proses inisialisasi, transfer data, dan eksekusi kernel pada OpenCL masih lebih besar dibandingkan keuntungan paralelisme yang diperoleh pada beban kerja yang relatif kecil.

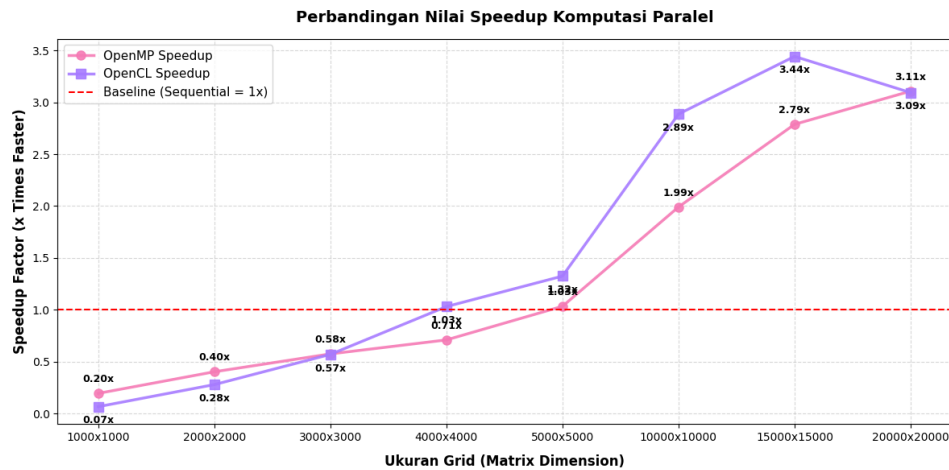
Ketika ukuran *grid* mencapai 10000×10000 hingga 20000×20000 , implementasi paralel mulai menunjukkan keunggulannya. Pada grid 10000×10000 , OpenMP dan OpenCL berhasil menurunkan waktu eksekusi menjadi 8,2206 detik dan 5,6704 detik dibandingkan 16,3672 detik pada implementasi sekuensial. Hasil ini menunjukkan bahwa semakin besar ukuran *grid* yang diproses, semakin efektif pemanfaatan paralelisme yang tersedia. Secara umum, OpenCL memberikan performa terbaik pada sebagian besar grid berukuran besar, meskipun pada ukuran 20000×20000 waktu eksekusi OpenMP dan OpenCL menjadi relatif sama.

5.2.2. Hasil Benchmarking Speedup

Nilai *speedup* digunakan untuk merepresentasikan rasio performa komputasi paralel, atau untuk melihat berapa kali lipat program berjalan lebih cepat dibandingkan dengan performa sekuensial tunggal sebagai *baseline* ($1 \times$).

Ukuran Grid	OpenMP	OpenCL
1000×1000	0.1951	0.0671
2000×2000	0.4032	0.2796
3000×3000	0.5754	0.5706
4000×4000	0.7102	1.0312
5000×5000	1.034	1.325
10000×10000	1.991	2.8863
15000×15000	2.7875	3.4419
20000×20000	3.1074	3.093

Tabel 4. Hasil Speedup



Gambar 6. Grafik Perbandingan Nilai Speedup Komputasi Paralel

Berdasarkan Tabel dan visualisasi diatas, nilai *speedup* OpenMP dan OpenCL mengalami peningkatan seiring bertambahnya ukuran *grid*. Pada ukuran *grid* 1000×1000 sampai 3000×3000 , nilai *speedup* kedua metode masih berada di bawah 1, yang menunjukkan bahwa implementasi paralel belum mampu melebihi implementasi sekuensial. Kondisi ini disebabkan oleh *overhead* paralelisasi yang masih mendominasi waktu komputasi sehingga keuntungan dari penggunaan banyak *thread* atau *work-item* belum dapat dimanfaatkan secara optimal.

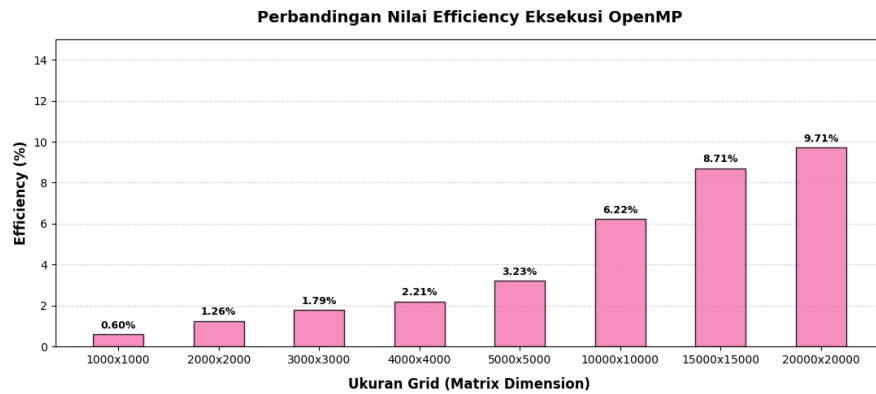
Ketika ukuran *grid* semakin besar, nilai *speedup* meningkat secara signifikan. OpenMP dan OpenCL mulai mencapai *speedup* di atas 1 pada *grid* 4000×4000 . Nilai *speedup* tertinggi diperoleh pada *grid* 15000×15000 , yaitu sebesar 2,7875 untuk OpenMP dan 3,4419 untuk OpenCL. Hasil ini menunjukkan bahwa implementasi OpenCL mampu memanfaatkan paralelisme GPU dengan lebih efektif dibandingkan OpenMP pada sebagian besar skenario pengujian, terutama ketika beban kerja yang diproses semakin besar

5.2.3. Hasil Benchmarking Efficiency

Analisis efisiensi bertujuan untuk mengukur seberapa efektif pemanfaatan kapasitas inti prosesor (*hardware cores*) yang dialokasikan selama eksekusi paralel berjalan. Nilai ini didapatkan dari hasil pembagian nilai *speedup* dengan jumlah total *thread* aktif pada perangkat keras.

Ukuran Grid	OpenMP
1000×1000	0.006
2000×2000	0.0126
3000×3000	0.0179
4000×4000	0.0221
5000×5000	0.0323
10000×10000	0.0622
15000×15000	0.0871
20000×20000	0.0971

Tabel 5. Hasil Efficiency



Gambar 7. Grafik Perbandingan Nilai Efficiency Eksekusi OpenMP

Berdasarkan Tabel 5 dan Gambar 7, nilai *efficiency* OpenMP menunjukkan tren peningkatan seiring bertambahnya ukuran *grid*. Pada *grid* berukuran kecil, nilai *efficiency* masih sangat rendah, yaitu hanya 0,006 pada ukuran 1000×1000 . Nilai tersebut menunjukkan bahwa sebagian besar sumber daya komputasi yang tersedia belum dapat dimanfaatkan secara efektif karena jumlah pekerjaan yang diproses masih terlalu kecil dibandingkan jumlah thread yang digunakan.

Seiring meningkatnya ukuran *grid*, nilai *efficiency* juga meningkat hingga mencapai 0,0971 pada ukuran 20000×20000 . Peningkatan ini menunjukkan bahwa semakin besar beban kerja yang diberikan, semakin efektif pemanfaatan thread OpenMP. Meskipun demikian, nilai *efficiency* yang diperoleh masih jauh dari kondisi ideal karena adanya overhead sinkronisasi, operasi atomik, serta ketidakseimbangan beban kerja yang terjadi selama proses eksplorasi BFS, sehingga tidak seluruh thread dapat bekerja secara optimal sepanjang waktu eksekusi.

5.3. Analisis Faktor Non Linearitas Percepatan

Hasil pengujian menunjukkan bahwa peningkatan performa yang diperoleh melalui OpenMP dan OpenCL tidak bersifat linier terhadap pertambahan ukuran *grid*. Meskipun implementasi paralel mampu menghasilkan percepatan yang signifikan pada *grid* berukuran besar, terdapat berbagai faktor yang membatasi pencapaian *speedup* ideal. Faktor-faktor tersebut meliputi *overhead* komunikasi data, keterbatasan *bandwidth* memori, ketidakseimbangan beban kerja, serta biaya sinkronisasi yang muncul selama proses komputasi paralel.

5.3.1. Komunikasi Data dan Latensi Bus Interkoneksi (Overhead Latency)

Pada implementasi OpenCL, data *grid*, *frontier*, dan struktur BFS lainnya harus dipindahkan terlebih dahulu dari memori *host* (CPU) ke memori *device* (GPU) sebelum proses komputasi dapat dilakukan. Setelah proses BFS selesai, hasil komputasi juga perlu dikembalikan ke *host* untuk diproses lebih lanjut. Proses transfer data ini menimbulkan *overhead* tambahan yang tidak ditemukan pada implementasi sekuensial maupun OpenMP.

Dampak *overhead* tersebut terlihat jelas pada ukuran *grid* kecil hingga menengah. Pada *grid* 1000×1000 , OpenCL membutuhkan waktu 1,4793 detik, jauh lebih lambat dibandingkan implementasi sekuensial yang hanya membutuhkan 0,0993 detik. Hal ini menunjukkan bahwa waktu yang diperlukan untuk transfer data dan inisialisasi kernel lebih besar dibandingkan waktu komputasi aktual yang dilakukan oleh GPU. Akibatnya, keuntungan paralelisme GPU belum mampu menutupi biaya komunikasi data yang muncul [23].

5.3.2. Hambatan Bandwidth Memori (Memory Bottleneck)

Algoritma BFS termasuk algoritma yang cenderung bersifat *memory-bound* karena sebagian besar waktu eksekusi digunakan untuk mengakses data pada memori dibandingkan melakukan operasi komputasi yang kompleks. Selama proses eksplorasi *frontier*, setiap *thread* atau *work-item* harus membaca status *visited*, memeriksa isi *grid*, dan memperbarui *frontier* berikutnya secara berulang [10].

Ketika ukuran *grid* meningkat hingga puluhan juta simpul, jumlah akses memori juga meningkat secara signifikan. Kondisi ini dapat menyebabkan *bandwidth* memori menjadi *bottleneck* yang membatasi performa sistem. Pada implementasi OpenCL, meskipun GPU memiliki jumlah unit pemrosesan yang sangat besar, seluruh *work-item* tetap harus berbagi *bandwidth* memori yang tersedia. Akibatnya, peningkatan jumlah pekerjaan tidak selalu menghasilkan peningkatan performa yang proporsional karena akses memori menjadi faktor pembatas utama [24].

5.3.3. Ketidakseimbangan Beban Kerja Multi-Thread (Load Imbalance)

Pada implementasi OpenMP, pembagian pekerjaan dilakukan berdasarkan simpul-simpul yang terdapat pada *frontier* saat ini. Namun, jumlah tetangga yang harus diperiksa oleh setiap simpul tidak selalu sama. Beberapa *thread* mungkin memperoleh bagian pekerjaan yang lebih banyak, sedangkan *thread* lainnya menyelesaikan tugas lebih cepat dan harus menunggu proses sinkronisasi berikutnya [24].

Fenomena *load imbalance* ini menyebabkan sebagian *thread* tidak dapat dimanfaatkan secara optimal selama seluruh waktu eksekusi. Dampaknya terlihat pada nilai *efficiency* OpenMP yang relatif rendah meskipun ukuran *grid* terus diperbesar. Meskipun *speedup* meningkat hingga mencapai 3,1074 pada *grid* 20000×20000 , nilai *efficiency* hanya mencapai sekitar 9,71%, yang menunjukkan bahwa sebagian besar thread tidak bekerja secara maksimal sepanjang proses komputasi.

5.3.4. Biaya Sinkronisasi Global dan Operasi Atomik

Baik OpenMP maupun OpenCL menggunakan mekanisme sinkronisasi untuk menjaga konsistensi data selama proses eksplorasi BFS. Pada OpenMP digunakan operasi atomik untuk memperbarui status *visited*, sedangkan pada OpenCL digunakan operasi atomik dan *barrier* untuk memastikan sinkronisasi antar *work-item* [14], [15].

Meskipun diperlukan untuk mencegah *race condition*, operasi sinkronisasi tersebut menambah *overhead* komputasi. Ketika banyak *thread* atau *work-item* mencoba mengakses lokasi memori yang sama secara bersamaan, akan terjadi *contention* yang menyebabkan

sebagian unit pemrosesan harus menunggu giliran untuk melakukan pembaruan data. Semakin besar *frontier* yang diproses, semakin tinggi frekuensi operasi atomik yang dilakukan, sehingga sebagian keuntungan paralelisme menjadi berkurang akibat biaya sinkronisasi tersebut.

5.3.5. Overlap dan Overhead pada Ukuran Grid Kecil

Pada ukuran *grid* kecil, implementasi sekuensial menunjukkan performa yang lebih baik dibandingkan OpenMP maupun OpenCL. Hal ini terjadi karena waktu komputasi yang diperlukan untuk menyelesaikan BFS relatif singkat sehingga *overhead* paralelisasi menjadi faktor dominan dalam total waktu eksekusi.

Khusus pada OpenCL, overhead tersebut mencakup proses alokasi *buffer*, transfer data melalui jalur PCIe, kompilasi kernel, eksekusi kernel, serta pembacaan kembali hasil komputasi ke CPU. Sementara itu pada OpenMP, *overhead* muncul akibat pembentukan *thread* dan proses sinkronisasi antar *thread*. Oleh karena itu, pada *grid* berukuran kecil keuntungan paralelisme belum mampu mengimbangi biaya tambahan tersebut, sehingga implementasi *sekuensial* tetap menjadi pilihan yang lebih efisien [25], [26].

5.3.6. Efisiensi dan Skalabilitas Sistem

Berdasarkan hasil pengujian, baik OpenMP maupun OpenCL menunjukkan peningkatan performa ketika ukuran grid diperbesar. OpenMP yang awalnya lebih lambat dibandingkan implementasi sekuensial mulai menghasilkan *speedup* lebih besar dari satu pada *grid* 10000×10000 . Sementara itu, OpenCL telah mencapai *speedup* di atas satu sejak ukuran grid 4000×4000 dan mencapai nilai tertinggi sebesar 3,4419 pada *grid* 15000×15000 .

Meskipun demikian, peningkatan *speedup* yang diperoleh tidak bersifat linier. Jika skalabilitas sistem bersifat ideal, maka penambahan ukuran masalah dan pemanfaatan lebih banyak unit pemrosesan seharusnya menghasilkan peningkatan *speedup* yang sebanding. Pada praktiknya, berbagai faktor seperti *memory bottleneck*, *load imbalance*, *overhead* sinkronisasi, dan biaya komunikasi data membatasi percepatan maksimum yang dapat dicapai. Hasil ini sejalan dengan konsep *Amdahl's Law* yang menyatakan bahwa keberadaan bagian program yang tidak dapat diparalelkan akan membatasi *speedup* total sistem. Oleh karena itu, meskipun OpenMP dan OpenCL mampu meningkatkan performa BFS secara signifikan pada *grid* berukuran besar, peningkatan tersebut tetap memiliki batas yang ditentukan oleh karakteristik algoritma dan arsitektur perangkat keras yang digunakan [20].

5.4. Analisis Kelebihan Sistem

5.4.1. Implementasi Tiga Pendekatan Komputasi dalam Satu Sistem

Salah satu kelebihan utama sistem ini adalah kemampuannya untuk mengintegrasikan tiga pendekatan komputasi yang berbeda, yaitu BFS sekuensial, OpenMP, dan OpenCL, dalam satu lingkungan pengujian yang sama. Integrasi ini memungkinkan proses perbandingan performa dilakukan secara langsung dengan menggunakan data *input*, ukuran *grid*, dan kondisi

lingkungan yang identik sehingga hasil *benchmarking* menjadi lebih konsisten dan objektif. Selain itu, keberadaan implementasi sekuensial sebagai *baseline* mempermudah proses evaluasi peningkatan performa yang diperoleh melalui penerapan teknik paralelisme.

5.4.2. Kemampuan Menangani Grid Berskala Besar

Sistem mampu menjalankan proses pencarian jalur pada *grid* dengan ukuran yang sangat besar hingga 20000×20000 sel. Kemampuan ini menunjukkan bahwa implementasi yang dikembangkan memiliki skalabilitas yang baik dalam menangani peningkatan jumlah simpul yang harus dieksplorasi. Berdasarkan hasil pengujian, seluruh metode berhasil menyelesaikan proses pencarian jalur pada seluruh ukuran *grid* yang diuji, sehingga sistem dapat digunakan untuk mengevaluasi karakteristik performa BFS pada berbagai tingkat kompleksitas permasalahan.

5.4.3. Pemanfaatan Paralelisme CPU melalui OpenMP

Implementasi OpenMP memungkinkan sistem memanfaatkan kemampuan *multi-core processor* untuk mempercepat proses eksplorasi *frontier* BFS. Penggunaan *thread-local frontier* dan operasi atomik berhasil menjaga konsistensi data sekaligus mengurangi konflik akses terhadap struktur data bersama. Hasil pengujian menunjukkan bahwa performa OpenMP meningkat secara signifikan pada ukuran *grid* yang besar, dengan nilai *speedup* mencapai 3,1074 kali pada *grid* 20000×20000 dibandingkan implementasi sekuensial.

5.4.4. Pemanfaatan Paralelisme GPU melalui OpenCL

Implementasi OpenCL memungkinkan sistem memanfaatkan ribuan *work-item* GPU untuk melakukan eksplorasi simpul secara paralel. Pendekatan ini terbukti memberikan keuntungan yang lebih besar dibandingkan OpenMP pada sebagian besar grid berukuran besar. Berdasarkan hasil pengujian, OpenCL menghasilkan *speedup* tertinggi sebesar 3,4419 kali pada *grid* 15000×15000 dan memperoleh waktu eksekusi terendah pada sebagian besar skenario pengujian berskala besar, menunjukkan efektivitas GPU dalam menangani beban kerja BFS yang bersifat *data-intensive*.

5.4.5. Validasi Efektivitas Komputasi Paralel dan Heterogen

Hasil pengujian menunjukkan bahwa sistem berhasil memperlihatkan perbedaan karakteristik antara komputasi sekuensial, paralel CPU, dan komputasi heterogen berbasis GPU. Pada *grid* kecil, implementasi sekuensial masih lebih unggul karena *overhead* paralelisme, sedangkan pada *grid* besar OpenMP dan terutama OpenCL mampu memberikan peningkatan performa yang signifikan. Temuan ini menunjukkan bahwa sistem yang telah dikembangkan berhasil digunakan sebagai cara untuk mengetahui pengaruh ukuran masalah terhadap efektivitas berbagai pendekatan komputasi paralel dan heterogen dalam menyelesaikan permasalahan *pathfinding* berbasis BFS.

5.5. Analisis Keterbatasan Sistem

5.5.1. Implementasi BFS

Sistem yang dikembangkan menggunakan algoritma BFS yang dirancang untuk menemukan lintasan terpendek pada graf tanpa bobot. Akibatnya, sistem kurang cocok untuk menangani kasus pencarian jalur yang memiliki biaya perpindahan berbeda pada setiap simpul atau sisi. Pada lingkungan yang memerlukan pertimbangan bobot jalur, algoritma lain seperti A* umumnya lebih sesuai digunakan dibandingkan BFS [27], [28].

5.5.2. Representasi Lingkungan Pencarian

Lingkungan pencarian direpresentasikan dalam bentuk *grid* dua dimensi dengan pergerakan terbatas pada arah tertentu dan *obstacle* yang dibuat secara acak. Representasi ini belum mampu menggambarkan kondisi lingkungan yang lebih kompleks seperti graf tidak beraturan, peta dunia nyata, atau lingkungan tiga dimensi. Oleh karena itu, hasil pengujian yang diperoleh masih terbatas pada karakteristik *grid* yang digunakan dalam sistem ini.

5.5.3. Overhead pada Implementasi OpenMP

Meskipun OpenMP mampu memanfaatkan banyak inti prosesor secara bersamaan, performa yang diperoleh masih dipengaruhi oleh *overhead* pembentukan *thread*, sinkronisasi, dan operasi atomik. Pada ukuran *grid* kecil hingga menengah, *overhead* tersebut bahkan lebih besar dibandingkan keuntungan yang diperoleh dari paralelisme sehingga waktu eksekusi OpenMP menjadi lebih lambat daripada implementasi sekuensial.

5.5.4. Overhead Transfer Data pada OpenCL

Implementasi OpenCL memerlukan proses transfer data antara CPU dan GPU sebelum dan sesudah eksekusi kernel dilakukan. Proses transfer ini menimbulkan *overhead* tambahan yang cukup signifikan, terutama pada ukuran *grid* kecil. Akibatnya, keuntungan paralelisme GPU tidak selalu dapat langsung dirasakan karena sebagian waktu eksekusi digunakan untuk proses komunikasi data melalui jalur interkoneksi antara CPU dan GPU [25].

5.5.5. Keterbatasan Skalabilitas Paralel

Hasil pengujian menunjukkan bahwa peningkatan performa yang diperoleh tidak bersifat linear terhadap jumlah sumber daya komputasi yang digunakan. Nilai *speedup* dan *efficiency* masih dibatasi oleh faktor-faktor seperti bagian program yang bersifat sekuensial, hambatan *bandwidth* memori, ketidakseimbangan beban kerja, serta biaya sinkronisasi selama proses eksplorasi BFS. Oleh karena itu, penambahan *thread* CPU atau *work-item* GPU tidak selalu menghasilkan percepatan yang proporsional.

5.5.6. Belum Memanfaatkan Optimasi GPU Tingkat Lanjut

Implementasi OpenCL yang digunakan masih berfokus pada pemanfaatan paralelisme dasar melalui *work-item* dan operasi atomik. Sistem belum menerapkan berbagai teknik optimasi GPU tingkat lanjut seperti penggunaan *local memory* secara intensif, *memory coalescing*, *kernel fusion*, atau teknik *load balancing* yang lebih adaptif. Akibatnya, performa yang diperoleh kemungkinan belum merepresentasikan potensi maksimum yang dapat dicapai oleh perangkat GPU yang digunakan [15], [29].

BAB VI. KESIMPULAN DAN SARAN

Proyek ini mengimplementasikan algoritma *Breadth-First Search* (BFS) untuk permasalahan *pathfinding* pada *grid* menggunakan tiga pendekatan komputasi yang berbeda, yaitu implementasi sekuensial, paralel CPU dengan OpenMP, dan paralel GPU dengan OpenCL. Berdasarkan hasil pengujian yang dilakukan pada berbagai ukuran *grid*, mulai dari 1000×1000 hingga 20000×20000 , seluruh implementasi mampu menemukan lintasan terpendek secara benar dan konsisten. Hasil benchmarking menunjukkan bahwa performa ketiga metode sangat dipengaruhi oleh ukuran *grid* yang diproses. Pada ukuran *grid* kecil, implementasi sekuensial memberikan waktu eksekusi terbaik karena tidak mengalami overhead paralelisasi. Namun, ketika ukuran *grid* semakin besar, implementasi OpenMP dan OpenCL mampu memanfaatkan paralelisme untuk mengurangi waktu komputasi secara signifikan. Nilai *speedup* yang terus meningkat seiring bertambahnya ukuran *grid* menunjukkan bahwa pendekatan paralel dan heterogen efektif digunakan untuk mempercepat proses eksplorasi BFS pada permasalahan berskala besar.

Dalam proses pengembangan dan implementasinya, terdapat beberapa kendala yang membatasi akselerasi maksimum sistem. Salah satu kendala utama pada implementasi OpenCL adalah *overhead transfer* data antara *host* dan *device* yang muncul ketika struktur data BFS harus dipindahkan melalui jalur PCIe sebelum dan sesudah eksekusi kernel. Selain itu, algoritma BFS memiliki karakteristik *memory-bound* sehingga performanya sangat dipengaruhi oleh *bandwidth* memori dan pola akses data. Pada implementasi OpenMP, ketidakseimbangan beban kerja (*load imbalance*) menyebabkan sebagian *thread* menyelesaikan pekerjaannya lebih cepat dibandingkan *thread* lainnya sehingga pemanfaatan sumber daya komputasi menjadi kurang optimal. Sementara itu, penggunaan mekanisme sinkronisasi seperti operasi atomik pada OpenMP, serta *barrier* dan *atomic_cmpxchg* pada OpenCL, meskipun diperlukan untuk mencegah *race condition* dan menjaga konsistensi data, menimbulkan biaya latensi tambahan yang mengurangi efisiensi paralelisme secara keseluruhan.

Untuk pengembangan di masa mendatang, terdapat beberapa optimasi yang dapat diterapkan untuk meningkatkan performa sistem. Pada implementasi OpenCL, penggunaan *local memory* sebagai *cache* sementara untuk menyimpan data tetangga *grid* dan status *frontier* dapat mengurangi jumlah akses ke *global memory* yang memiliki latensi lebih tinggi. Teknik *memory coalescing* dan pemanfaatan *shared local memory* secara lebih intensif juga berpotensi meningkatkan throughput akses data pada GPU. Selain itu, strategi *load balancing* yang lebih adaptif dapat diterapkan pada implementasi OpenMP untuk mengurangi ketidakseimbangan beban kerja antar *thread*. Pengembangan lebih lanjut juga dapat dilakukan dengan mengimplementasikan algoritma *pathfinding* lain untuk menangani graf berbobot, serta melakukan pengujian pada lingkungan yang lebih kompleks.

Secara keseluruhan, hasil pengujian menunjukkan bahwa implementasi OpenCL merupakan metode yang paling optimal untuk kasus BFS pada *grid* berukuran besar yang diuji dalam penelitian ini. OpenCL berhasil memperoleh waktu eksekusi tercepat pada sebagian besar skenario berskala besar dan mencapai nilai *speedup* tertinggi sebesar 3,4419 kali pada

grid 15000×15000 . OpenMP juga menunjukkan peningkatan performa yang signifikan dibandingkan implementasi sekuensial dengan *speedup* maksimum sebesar 3,1074 kali pada *grid* 20000×20000 , meskipun masih dibatasi oleh overhead sinkronisasi dan load imbalance. Sementara itu, implementasi sekuensial tetap menjadi pilihan yang paling efisien untuk grid berukuran kecil karena tidak mengalami biaya paralelisasi. Dengan demikian, dapat disimpulkan bahwa efektivitas metode komputasi sangat bergantung pada ukuran masalah yang diproses, di mana OpenCL menjadi pilihan terbaik untuk beban kerja BFS berskala besar, sedangkan implementasi sekuensial lebih sesuai untuk kasus dengan ukuran grid yang relatif kecil.

LAMPIRAN

[Link video penjelasan proyek](#)

[Link repository GitHub](#)

DAFTAR PUSTAKA

- [1] Y. Sun, Q. Yuan, Q. Gao, and L. Xu, "A Multiple Environment Available Path Planning Based on an Improved A* Algorithm," *International Journal of Computational Intelligence Systems* 2024 17:1, vol. 17, no. 1, pp. 172-, Jul. 2024, doi: 10.1007/S44196-024-00571-Z.
- [2] S. R. Lawande, G. Jasmine, J. Anbarasi, and L. I. Izhar, "A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games," *Applied Sciences (Switzerland)*, vol. 12, no. 11, Jun. 2022, doi: 10.3390/app12115499.
- [3] C. Zhang, Q. Hu, J. Tu, Y. Dong, Y. Tan, and Y. He, "A*-plus path planning algorithm for large-scale grid maps," *Array*, vol. 28, p. 100593, Dec. 2025, doi: 10.1016/J.ARRAY.2025.100593.
- [4] J. Li, "Efficient parallelism in Breadth-First Search: A comprehensive analysis and implementation," *Applied and Computational Engineering*, vol. 36, no. 1, pp. 185–191, Feb. 2024, doi: 10.54254/2755-2721/36/20230443.
- [5] M. Barkhordar *et al.*, "ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System".
- [6] "(PDF) Performance Improvement using Optimal Thread Allocation Algorithm in Multicore Processor." Accessed: Jun. 23, 2026. [Online]. Available: https://www.researchgate.net/publication/343236203_Performance_Improvement_using_Optimal_Thread_Allocation_Algorithm_in_Multicore_Processor

- [7] X. Yan, X. Shi, L. Wang, and H. Yang, "An OpenCL micro-benchmark suite for GPUs and CPUs," *Journal of Supercomputing*, vol. 69, no. 2, pp. 693–713, Aug. 2014, doi: 10.1007/S11227-014-1112-2.
- [8] K. Thouti and S. R. S. -, "Comparison of OpenMP & OpenCL Parallel Processing Technologies," *International Journal of Advanced Computer Science and Applications*, vol. 3, no. 4, 2012, doi: 10.14569/IJACSA.2012.030410.
- [9] I. A. Stewart, T. P. Hage, Y.-C. Hsu, and M. J. Buehler, "GraphAgents: Knowledge Graph-Guided Agentic AI for Cross-Domain Materials Design," Feb. 2026, Accessed: Jun. 24, 2026. [Online]. Available: <https://arxiv.org/pdf/2602.07491v1>
- [10] D. Yoon and S. Oh, "SURF: Direction-Optimizing Breadth-First Search Using Workload State on GPUs," *Sensors (Basel)*, vol. 22, no. 13, p. 4899, Jul. 2022, doi: 10.3390/S22134899.
- [11] T. Ichimura *et al.*, "Heterogeneous computing in a strongly-connected CPU-GPU environment: fast multiple time-evolution equation-based modeling accelerated using data-driven approach".
- [12] P. Thoman, P. Zangerl, and T. Fahringer, "Static Compiler Analyses for Application-specific Optimization of Task-Parallel Runtime Systems," *Journal of Signal Processing Systems 2018 91:3*, vol. 91, no. 3, pp. 303–320, Apr. 2018, doi: 10.1007/S11265-018-1356-9.
- [13] L. M. Munguia, D. A. Bader, and E. Ayguade, "Task-based parallel breadth-first search in heterogeneous environments," *2012 19th International Conference on High Performance Computing, HiPC 2012*, 2012, doi: 10.1109/HIPC.2012.6507474.
- [14] "OpenMP Application Programming Interface".
- [15] "The OpenCL™ Specification." Accessed: Jun. 24, 2026. [Online]. Available: https://registry.khronos.org/OpenCL/specs/unified/html/OpenCL_API.html
- [16] "Lecture 20-Related Programming Models: OpenCL 2 Objective-To Understand the OpenCL programming model-basic concepts and data types-Kernel structure-Application programming interface-Simple examples".
- [17] J. E. Stone, D. Gohara, and G. Shi, "OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Comput. Sci. Eng.*, vol. 12, no. 3, p. 66, May 2010, doi: 10.1109/MCSE.2010.69.
- [18] D. Beyer, S. Löwe, and P. Wendler, "Reliable benchmarking: requirements and solutions," *International Journal on Software Tools for Technology Transfer 2017 21:1*, vol. 21, no. 1, pp. 1–29, Nov. 2017, doi: 10.1007/S10009-017-0469-Y.

- [19] D. Ene and V. I. Anireh, "Performance Evaluation of Parallel Algorithms," *SSRG International Journal of Computer Science and Engineering*, vol. 9, pp. 10–14, 2022, doi: 10.14445/23488387/IJCSE-V9I6P102.
- [20] G. Schryen, "Speedup and efficiency of computational parallelization: A unifying approach and asymptotic analysis".
- [21] A. Segura, J. M. Arnau, and A. Gonzalez, "Irregular accesses reorder unit: improving GPGPU memory coalescing for graph-based workloads," *The Journal of Supercomputing* 2022 79:1, vol. 79, no. 1, pp. 762–787, Jul. 2022, doi: 10.1007/S11227-022-04621-1.
- [22] M. Bhaskar and R. Kanakagiri, "Performance-Driven Optimization of Parallel Breadth-First Search," *Proceedings of*, vol. 1.
- [23] "CUDA Best Practices Guide — CUDA C++ Best Practices Guide 13.3 documentation." Accessed: Jun. 25, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- [24] Z. Jiang, T. Liu, S. Zhang, Z. Guan, M. Yuan, and H. You, "Fast and Efficient Parallel Breadth-First Search with Power-law Graph Transformation".
- [25] "Tuning Performance On the GPU." Accessed: Jun. 25, 2026. [Online]. Available: https://developer.apple.com/library/archive/documentation/Performance/Conceptual/OpenCL_MacProgGuide/TuningPerformanceOntheGPU/TuningPerformanceOntheGPU.html
- [26] D. Álvarez, K. Sala, M. Maroñas, A. Roca, and V. Beltran, "Advanced Synchronization Techniques for Task-based Runtime Systems; Advanced Synchronization Techniques for Task-based Runtime Systems," 2021, doi: 10.1145/3437801.3441601.
- [27] S. Edelkamp and S. Schrödl, "Introduction," *Heuristic Search*, pp. 3–46, 2012, doi: 10.1016/B978-0-12-372512-7.00001-8.
- [28] R. Kala, "Graph search-based motion planning," *Autonomous Mobile Robots*, pp. 201–253, 2024, doi: 10.1016/B978-0-443-18908-1.00008-X.
- [29] A. Segura, J. M. Arnau, and A. Gonzalez, "Irregular accesses reorder unit: improving GPGPU memory coalescing for graph-based workloads," *The Journal of Supercomputing* 2022 79:1, vol. 79, no. 1, pp. 762–787, Jul. 2022, doi: 10.1007/S11227-022-04621-1.